L2 – Requirements Engineering

Topics covered

- ✧ Requirements elicitation
- ✧ Requirements specification
  - Functional and non-functional requirements
- ✧ Requirements validation
- ✧ Requirements change

What is requirement engineering?

- **Requirements engineering** is the process of **establishing the services that a customer requires** from a system and the **constraints** under which it operates and is developed.



- **System requirements** are the descriptions of the system services and constraints that are **generated during the requirements engineering process**.



- Requirements may serve a **dual function**
  - May be the basis for **a bid for a contract** - therefore must be **open to interpretation**;
  - May be the basis for the **contract itself** - therefore must be **defined in detail**;

Types of requirement

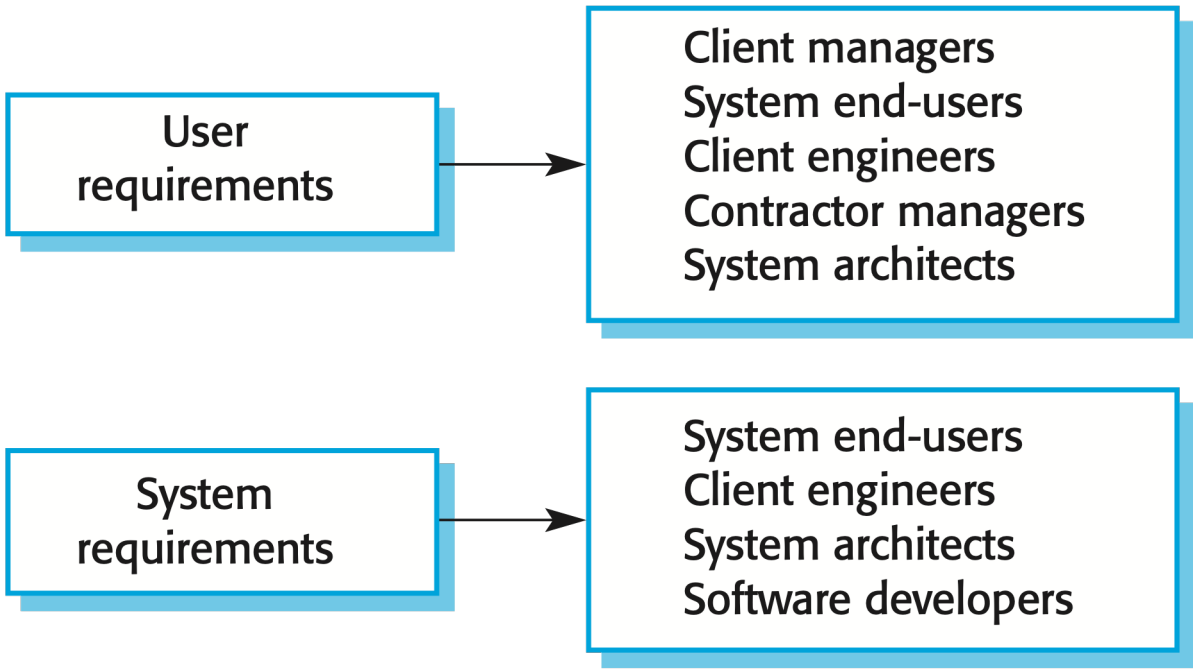
- User requirements
  - Statements in **natural language plus diagrams** of the services the system provides and its **operational constraints**.
  - Mainly written for customers.

> **1.** The Mentcare system shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

- System requirements
  - A **structured document** setting out **detailed descriptions** of the system's functions, services and operational constraints.

> **1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
> **1.2** The system shall generate the report for printing after 17.30 on the last working day of the month.
> **1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
> **1.4** If drugs are available in different dose units (e.g. 10mg, 20mg, etc) separate reports shall be created for each dose unit.
> **1.5** Access to drug cost reports shall be restricted to authorized users as listed on a management access control list.

Readers of requirements



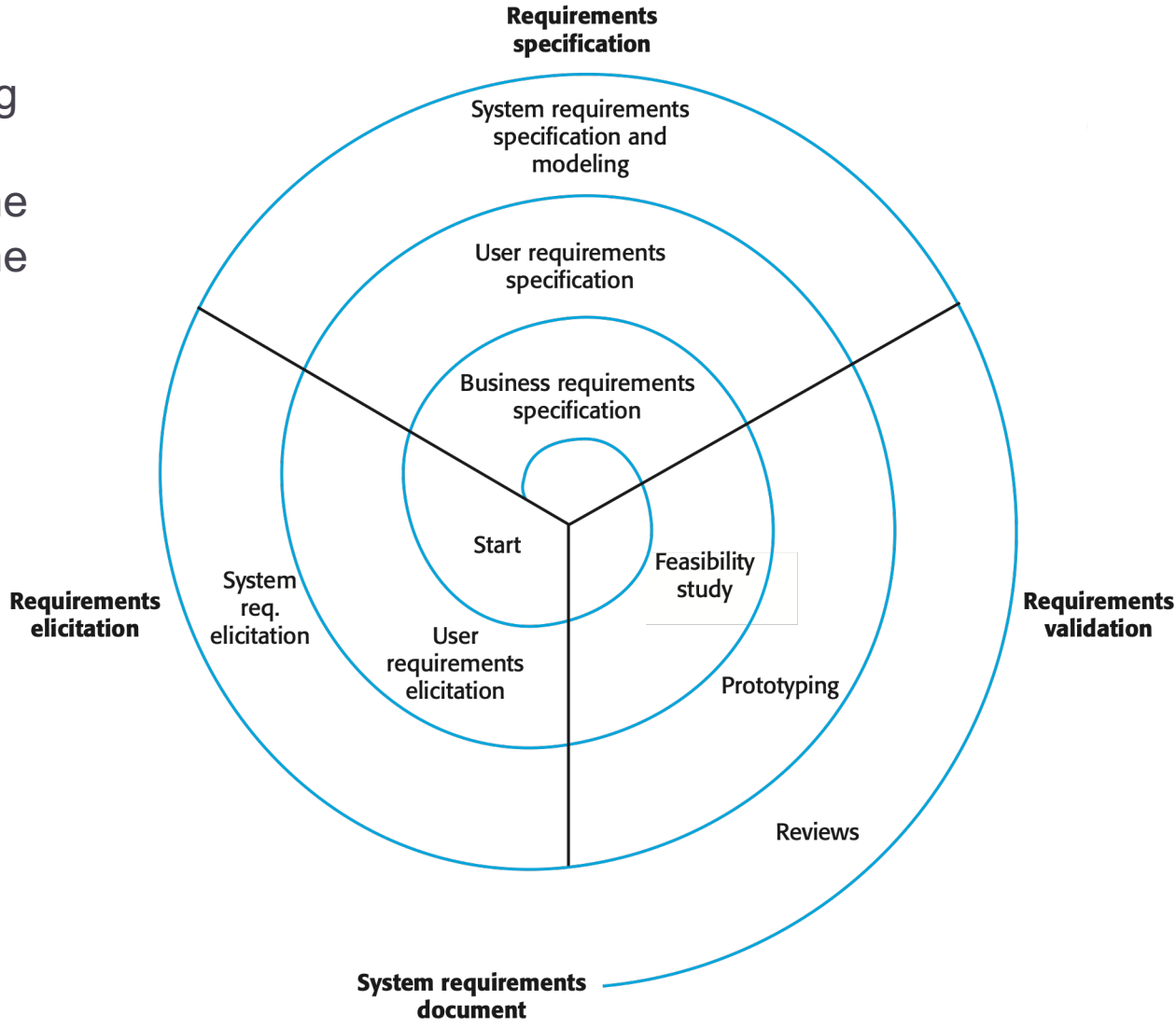
System stakeholders

- Any person or organization who either **affects or is affected by the system in some way** and so who has a legitimate **interest**
- Stakeholder types
  - End users
  - System managers
  - System owners
  - External stakeholders



A spiral view of the requirements engineering process

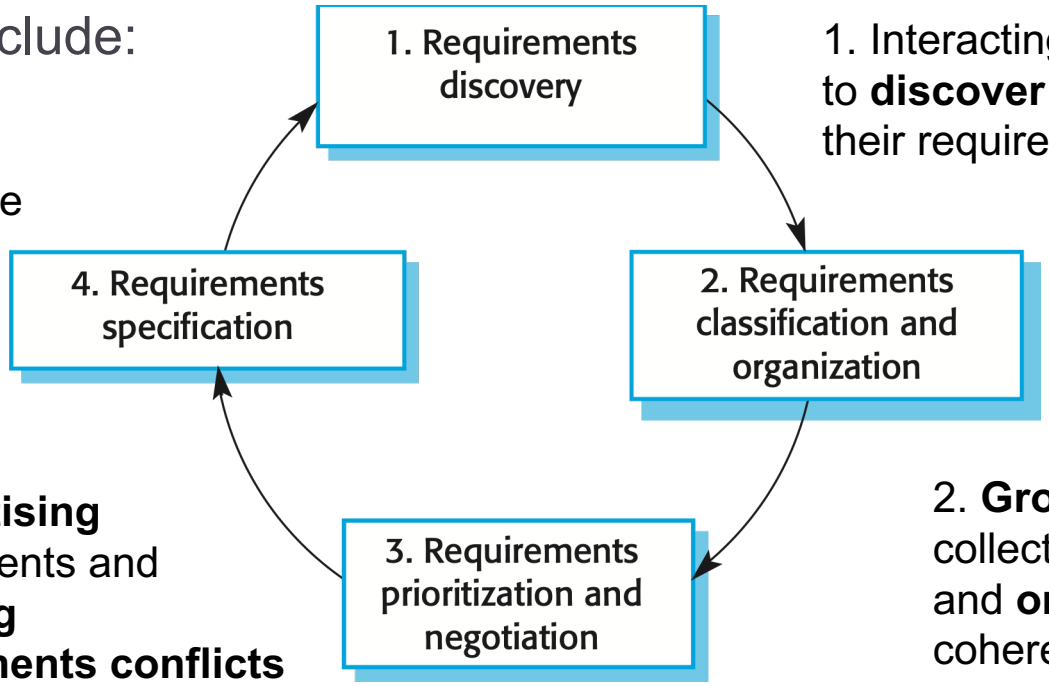
- The **processes used for RE vary widely** depending on the application domain, the people involved and the organisation developing the requirements. However, there are a number of generic activities common to all processes.
- In practice, RE is an **iterative activity** in which these processes are **interleaved**.



Requirements elicitation

Requirements elicitation and analysis

- Involves **technical staff** (software engineers) work with **customers** or a range of **system stakeholders** to find out about the application domain, the services that the system should provide, the required system performance, hardware constraints, other systems, etc.
- Stages include:



1. Interacting with stakeholders to **discover** and **understand** their requirements.

2. **Groups** unstructured collection of requirements and **organises** them into coherent clusters.

3. **Prioritising** requirements and **resolving requirements conflicts**

4. Requirements are **documented** and input into the next round of the spiral.

Problems of requirements elicitation

- Stakeholders **don't know** what they really want.
- Stakeholders express requirements in their **own terms**.
- Different stakeholders may have **conflicting requirements**.
- **Organisational and political factors** may influence the system requirements.
- The **requirements change** during the analysis process.
  - **New stakeholders** may emerge
  - the **business environment may change**

Requirements discovery

- The process of gathering information about the **required and existing systems**, and distilling the **user and system requirements** from this information.
- **Interaction is with system stakeholders** from managers to external regulators. Systems normally have a range of stakeholders.
- Methods:
  - Interviews
  - Ethnography / Observation
    - Spends a considerable time **observing and analysing how people actually work**.
    - People do **NOT** have to explain their work.
    - **Social and organisational factors** of importance may be observed.
    - Effective for understanding existing processes, but **CANNOT identify new features**

Interviewing

- **Formal or informal** interviews with stakeholders are part of most RE processes.
  - **Closed interviews** based on pre-determined list of questions
  - **Open interviews** where various issues are explored with stakeholders.
- Effective interviewing
  - **Be open-minded**, **avoid pre-conceived ideas** about the requirements and are **willing to listen to stakeholders**.
  - Prompt the interviewee to **get discussions** going using a springboard question, a requirements proposal, or by working together on a prototype system.

Observe-Share

This video shows a simulated client meeting (interview) conservation for a Band Guy and a Web Developer.

Give **one good act** and **one bad act** you find from the Web Developer on handling the client's inquiry.



Problems with interviews

- Application specialists may use language to describe their work that isn't easy for the requirements engineer to understand.
- Interviews are **not good for understanding domain requirements**
  - Requirements engineers cannot understand specific domain terminology;
  - Some domain knowledge is so familiar to stakeholders that they find it hard to articulate or think that it isn't worth articulating.

Requirements specification

User stories and scenarios

- User stories
  - **real-life examples** of how a system can be used
- Scenario
  - A description of how a system may be used for a **particular task**
  - A structured form of user story
- Because they are based on a **practical situation**, stakeholders can relate to them and **can comment on their situation with respect to the story**.

Example of User Story in the iLearn system - Photo sharing in the classroom

Jack is a primary school teacher in Ullapool (a village in northern Scotland). He has decided that a class project should be focused around the fishing industry in the area, looking at the history, development and economic impact of fishing. As part of this, pupils are asked to gather and share reminiscences from relatives, use newspaper archives and collect old photographs related to fishing and fishing communities in the area. Pupils use an iLearn wiki to gather together fishing stories and SCRAN (a history resources site) to access newspaper archives and photographs. However, Jack also needs a photo sharing site as he wants pupils to take and comment on each others' photos and to upload scans of old photographs that they may have in their families.

Jack sends an email to a primary school teachers group, which he is a member of, to see if anyone can recommend an appropriate system. Two teachers reply and both suggest that he uses KidsTakePics, a photo sharing site that allows teachers to check and moderate content. As KidsTakePics is not integrated with the iLearn authentication service, he sets up a teacher and a class account. He uses the iLearn setup service to add KidsTakePics to the services seen by the pupils in his class so that when they log in, they can immediately use the system to upload photos from their mobile devices and class computers.

Example of Scenario for uploading photos in the KidsTakePics system

Initial assumption: A user or a group of users have one or more digital photographs to be uploaded to the picture sharing site. These are saved on either a tablet or laptop computer. They have successfully logged on to KidsTakePics.

Normal: The user chooses upload photos and they are prompted to select the photos to be uploaded on their computer and to select the project name under which the photos will be stored. They should also be given the option of inputting keywords that should be associated with each uploaded photo. Uploaded photos are named by creating a conjunction of the user name with the filename of the photo on the local computer.

On completion of the upload, the system automatically sends an email to the project moderator asking them to check new content and generates an on-screen message to the user that this has been done.

What can go wrong: No moderator is associated with the selected project. An email is automatically generated to the school administrator asking them to nominate a project moderator. Users should be informed that there could be a delay in making their photos visible.

Photos with the same name have already been uploaded by the same user. The user should be asked if they wish to re-upload the photos with the same name, rename the photos or cancel the upload. If they chose to re-upload the photos, the originals are overwritten. If they chose to rename the photos, a new name is automatically generated by adding a number to the existing file name.

Other activities: The moderator may be logged on to the system and may approve photos as they are uploaded.

System state on completion: User is logged on. The selected photos have been uploaded and assigned a status 'awaiting moderation'. Photos are visible to the moderator and to the user who uploaded them.

Requirements specification

- The process of **writing down the user and system requirements** in a requirements document.
- User requirements have to be **understandable** by end-users and customers who do not have a technical background.
- **System requirements** are more **detailed requirements** and may include **more technical information**.
- The requirements may be **part of a contract** for the system development
  - It is therefore important that these are as **complete** as possible.

Ways of writing a system requirements specification

Notation	Description
Natural language	The requirements are written using **numbered sentences in natural language**. Each sentence should express one requirement.
Structured natural language	The requirements are written in natural language on a **standard form** or template. Each field provides information about an aspect of the requirement.
Graphical notations	**Graphical models, supplemented by text annotations**, are used to define the functional requirements for the system; UML use case and sequence diagrams are commonly used.
Mathematical specifications	These notations are based on **mathematical concepts such as finite-state machines or sets**. Although these unambiguous specifications can reduce the ambiguity in a requirements document, most customers don't understand a formal specification. They cannot check that it represents what they want and are reluctant to accept it as a system contract

Natural language specification

- Requirements are written as **natural language sentences**, **supplemented by diagrams and tables**.
- Used for writing requirements because it is **expressive, intuitive and universal**. This means that the requirements can be understood by users and customers.

3.2 The system shall measure the blood sugar and deliver insulin, if required, every 10 minutes. *(Changes in blood sugar are relatively slow so more frequent measurement is unnecessary; less frequent measurement could lead to unnecessarily high sugar levels.)*
3.6 The system shall run a self-test routine every minute with the conditions to be tested and the associated actions defined in Table 1. *(A self-test routine can discover hardware and software problems and alert the user to the fact the normal operation may be impossible.)*

Example of using natural language to write system requirements for the insulin pump software system

Guidelines for writing requirements

- Invent a **standard format** and use it for all requirements.
- Use language in a consistent way.
  - Use "**shall**" for mandatory requirements, which the system **must** support
  - Use "**should**" for desirable requirements, which are **not essential**
- Use **text highlighting** to identify key parts of the requirement.
- **Avoid** the use of **computer jargon**.
- Include an **explanation (rationale)** of why a requirement is necessary.

Problems with natural language

Lack of clarity — precision is hard without making the document unreadable.

Requirements confusion — functional and non-functional requirements get mixed up.

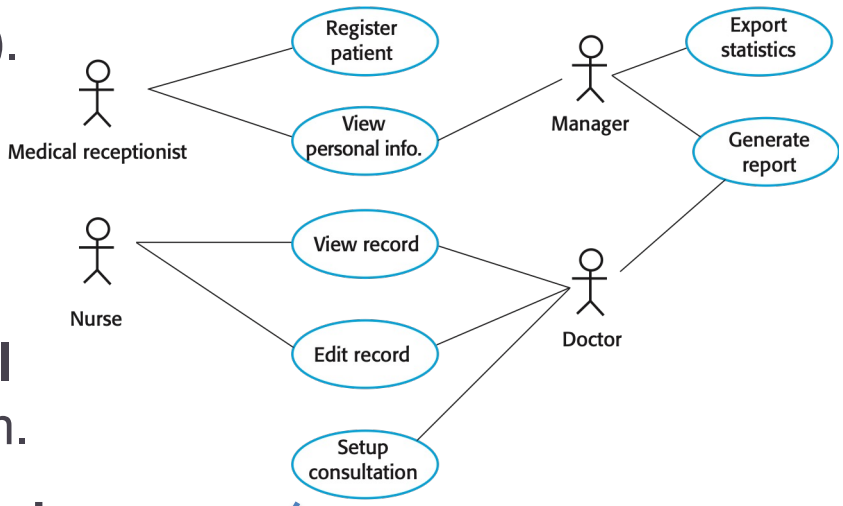
Requirements amalgamation — several different requirements expressed together.

One common approach: "The [feature or function] **shall/should** [do something based on user inputs and provide corresponding outputs]."

Use cases

- **A kind of scenario** that are included in the Unified Modeling Language (**UML**).
- Use cases identify the **actors** in an interaction and which describe the **interaction** itself.
- A set of use cases should **describe all possible interactions** with the system.
- **Each use case** should be **documented with a textual description**.
- High-level graphical model supplemented by **more detailed tabular description** .

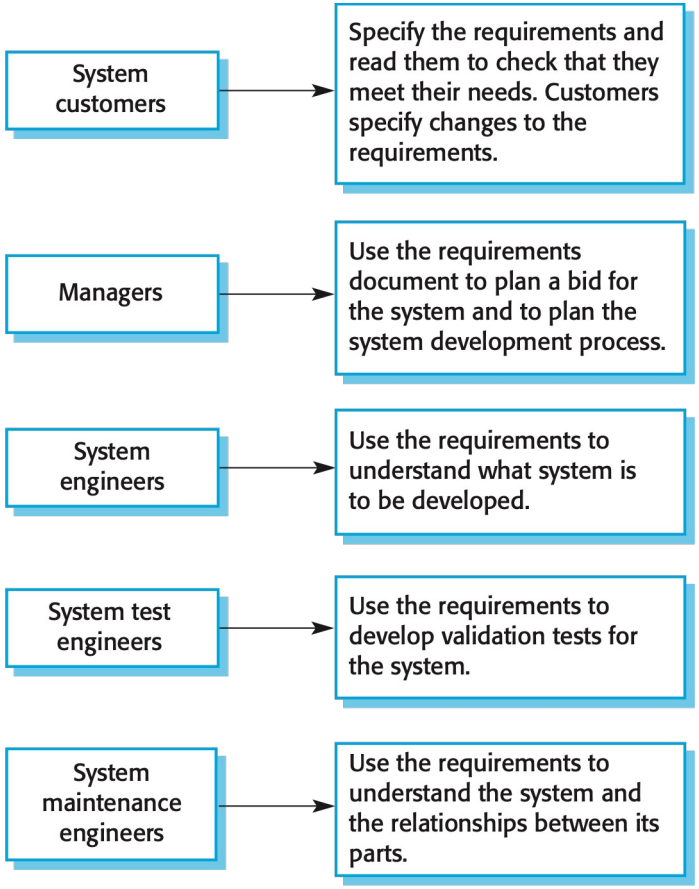
Example of using Use Cases to present system requirements for the Mentcare system



Setup consultation allows two or more doctors, working in different offices, to view the same patient record at the same time. One doctor initiates the consultation by choosing the people involved from a dropdown menu of doctors who are online. The patient record is then displayed on their screens, but only the initiating doctor can edit the record. In addition, a text chat window is created to help coordinate actions. It is assumed that a phone call for voice communication can be separately arranged.

The software requirements document

- The software requirements document (a.k.a. **Software Requirements Specification** or **SRS**) is the official statement of what is required of the system developers.
- Should include **both** a definition of **user requirements** and a specification of the **system requirements**.
- Example – "Requirements for the Mentcare system"(PDF)



Users of a requirements document

Functional and non-functional requirements

Functional and non-functional requirements

- Functional requirements
  - Statements of services the system **should provide**, how the system **should react** to particular inputs and how the system **should behave** in particular situations.
  - May state what the system **should not do**.
- Non-functional requirements
  - **Constraints on the services or functions** offered by the system such as timing constraints, constraints on the **development process, standards**, etc.
  - Often apply to the **system as a whole** rather than individual features or services.

Know more about …

- Domain requirements
  - Constraints on the system **from the domain of operation**

Functional requirements

- Describe **functionality** or **system services**.
- Depend on the **type of software**, **expected users** and the type of system where the software is used.
- **Functional user requirements** may be **high-level** statements of what the system should do.
- **Functional system requirements** should describe the system services **in detail.**

- **Examples in the Mentcare system:**
  - *"A user shall be able to search the appointments lists for all clinics."*
  - *"The system shall generate each day, for each clinic, a list of patients who are expected to attend appointments that day."*
  - *"Each staff member using the system shall be uniquely identified by his or her 8-digit employee number."*

General problems of functional requirements

✧ Requirements imprecision

- Problems arise when functional requirements are not precisely stated.
- **Ambiguous requirements** may be interpreted in **different ways** by users and developers. Consider the term 'search' in this requirement:

*"A user shall be able to **search** the appointment lists for all clinics."*

Search for a patient name across all appointments **in all clinics**

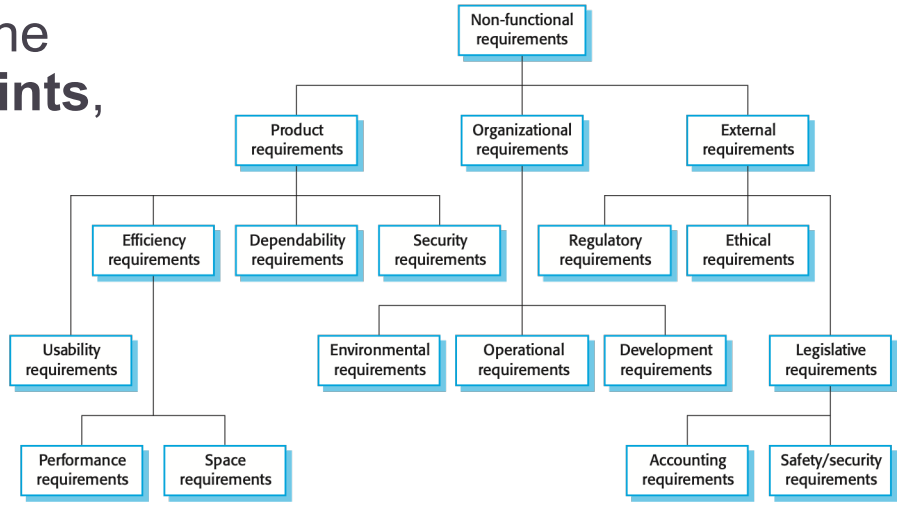
Search for a patient name **in an individual clinic.** User chooses clinic then search

✧ Requirements incompleteness and inconsistency

- In principle, requirements should be both complete and consistent.
  - **Complete:** They should include descriptions of all facilities required.
  - **Consistent:** There should be no conflicts or contradictions in the descriptions of the system facilities.

Non-functional requirements

- Non-functional requirements define **system properties and constraints**, e.g.
  - reliability,
  - response time,
  - storage requirements
  - I/O device capability,
  - data representations used in interfaces with other systems



Types of non-functional requirement

- **Process requirements** may also be specified mandating a particular IDE, programming language or development method.
- Non-functional requirements **may be more critical than functional requirements**. If these are not met, the system may be useless.

Examples of non-functional requirements in the Mentcare system

Product requirement
The Mentcare system shall be available to all clinics during normal working hours (Mon–Fri, 08:30–17:30). Downtime within normal working hours shall not exceed five seconds in any one day.

dependability

Organizational requirement
Users of the Mentcare system shall authenticate themselves using their health authority identity card.

operational

External requirement
The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

regulatory

Non-functional requirements implementation

- May **affect the overall architecture** of a system rather than the individual components.
  - For example, to ensure that performance requirements are met, you may have to organize the system to minimize communications between components.
- May **generate a number of related functional requirements** that define system services that are required.
  - It may also generate requirements that restrict existing requirements.

Metrics for non-functional requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Number of target systems

Goals and requirements

- **Problem**: Non-functional requirements may be very **difficult to state precisely** and imprecise requirements may be **difficult to verify**.
- **Goal**
  - A **general intention of the user**, such as “ease of use”, “the ability of the system to recover from failure”, “rapid user response”, or “errors are minimized”.
  - Goals are **helpful to developers** as they convey the intentions of the system users.
- **Verifiable non-functional requirement**
  - A statement **using some measure** that can be objectively **tested**.

Usability requirements

- **Goal**: The system should be **easy to use** by medical staff and should be organized in such a way that user **errors are minimized**.



Re-write

- **Testable non-functional requirement:** Medical staff shall be able to use all the system functions after four hours of training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use.

Requirements validation

Requirements validation

- Concerned with **demonstrating that the requirements** define the system that the customer really wants.
- **Requirements error costs are high** so validation is very important
- **Checking:**
  - **Validity** - Does the system provide the functions which **best support** customer's needs?
  - **Consistency** - Are there any requirements **conflicts**?
  - **Completeness** - Are all functions required by the customer **included**?
  - **Realism:** Can the requirements be implemented **given available budget & technology**?
  - **Verifiability** - Can the requirements be **checked**? Realistically **testable**?

Validation techniques

Requirements reviews

- Systematic manual analysis of the requirements.
- Both client and contractor should be involved in reviews.
- Reviews may be formal (with completed documents) or informal.

Prototyping

- Using an executable model of the system to check requirements.

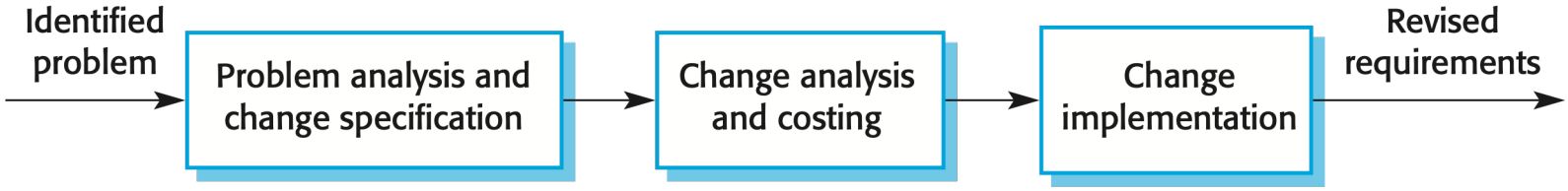
Test-case generation

- Developing tests for requirements to check testability.

Requirements change

Requirements management

- The process of managing changing requirements **during the requirements engineering process and system development**.
- Due to business, organizational and technical changes, new requirements emerge as a system is being developed and after it has gone into use.
- You need to **keep track of individual requirements and maintain links between dependent requirements** so that you can assess the impact of requirements changes. You need to establish a formal process for making change proposals and linking these to system requirements.
  - Project tracking software, e.g. Jira for SCRUM, may help.
- **Deciding** if a requirements change should be accepted:



Supplementary: AI & Workshop Preparation

User Stories: the Contract for Humans - and for AI

USER STORY — the client's own sentence: what they want, and why. This is what the client agrees to.

As a Staff Member, **I want** to choose one dish before the cut-off, **so that** my lunch is in the office order.

ACCEPTANCE CRITERIA — how we will TEST that the story is done. Written with the story, checked before it ships.

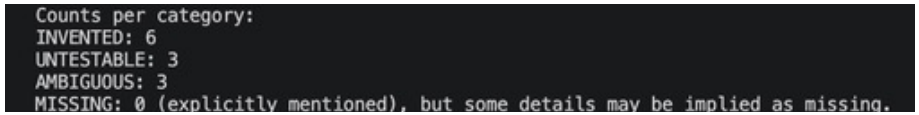
Given the time is before 10:30, **when** I choose a dish, **then** my order is saved.

The shape used in W2 and W3 (MealMate). "Acceptance criteria" is agile-practice vocabulary (XP user stories, L10).

- As a <role>, I want <goal>, so that <benefit> - plus **acceptance criteria that can be TESTED**.
- Your user stories now have **two readers**: your team, and your AI tools.
  - A precise story with acceptance criteria is also a **high-quality prompt**.
- **Weak story**: the AI fills the gaps with guesses. **Strong story**: the AI implements **YOUR intent**.
- Hands-on in W2: **Copilot drafts** the stories, **YOU review** them line by line - then a **second model audits** your review.

AI-Generated Requirements: What to Check

- **Hallucinated features** - plausible functions the client never asked for.
- **Untestable acceptance criteria** - "user-friendly", "fast", "secure" with **no measure**.
- **Ambiguity** - one story, several readings; would two developers build the same thing?
- **Missing cases** - errors, limits, empty states; AI drafts the **happy path** first.
- **Golden rule**: **AI drafts, engineers decide**. **Never forward an unread requirement**.



W2: the second model audits your review and counts exactly these categories (own screenshot).

References

Core reading

- Sommerville, Software Engineering, 10th ed. — Chapter 4 (Requirements Engineering).
- ISO/IEC/IEEE 29148 — Requirements engineering (standard overview).

Further reading & links

- Atlassian Agile Coach — user stories
- Cloudflare Workers AI (the W2 reviewer model)
- IEEE requirements standard (Sommerville)
- Requirements specification template (Wiegers)
- Characteristics of a Good Requirement
- Review Checklist for Software Requirements